UNIVERSITÉ LAVAL

Solutionnaire du TP1

À votre tour

0. Importer et compter

```
got_aretes <- read_csv("/home/public/reseaux/got_edges.csv")
got_sexe   <- read_csv("/home/public/reseaux/got_sex.csv")
```

```
nrow(got_aretes)
#> [1] 550
nrow(got_sexe)
#> [1] 127
persos <- unique(c(got_aretes$from, got_aretes$to))
length(persos)
#> [1] 127
setdiff(persos, got_sexe$name)
#> character(0)
setdiff(got_sexe$name, persos)
#> character(0)
```

550 liens, 127 personnages distincts, exactement ceux du fichier des sexes (les deux `setdiff()` sont vides). Compter seulement `from` aurait sous-estimé le nombre de personnages : 76 au lieu de 127.

1. Construire le graphe

```
g <- graph_from_data_frame(got_aretes)
g
#> IGRAPH 2013535 DNW- 127 550 --
#> + attr: name (v/c), weight (e/n)
#> + edges from 2013535 (vertex names):
#>  [1] Ned         ->Robert       Daenerys    ->Jorah
#>  [3] Jon         ->Sam          Ned         ->Littlefinger
#>  [5] Ned         ->Varys        Daenerys    ->Drogo
#>  [7] Ned         ->Arya         Catelyn     ->Robb
#>  [9] Bronn       ->Tyrion       Ned         ->Cersei
#> [11] Cersei      ->Robert       Littlefinger->Varys
#> [13] Shae        ->Tyrion       Ned         ->Catelyn
```

```
#> [15] Joffrey     ->Sansa         Ned         ->Pycelle
#> + ... omitted several edges
```

`DNW-` : **D**irigé, nœuds **N**ommés, liens pondérés (**W**, car une colonne s'appelle `weight`), non bipartite. 127 nœuds, 550 liens. Dirigé, parce que c'est la valeur par défaut, alors que l'ordre `from` / `to` ne veut rien dire ici.

```
got <- graph_from_data_frame(got_aretes, directed = FALSE, vertices = got_sexe)
got
#> IGRAPH 9f56d17 UNW- 127 550 --
#> + attr: name (v/c), sex (v/c), weight (e/n)
#> + edges from 9f56d17 (vertex names):
#>  [1] Ned         --Robert        Daenerys    --Jorah
#>  [3] Jon         --Sam           Ned         --Littlefinger
#>  [5] Ned         --Varys         Daenerys    --Drogo
#>  [7] Ned         --Arya          Catelyn     --Robb
#>  [9] Bronn       --Tyrion        Ned         --Cersei
#> [11] Cersei      --Robert        Littlefinger--Varys
#> [13] Shae        --Tyrion        Ned         --Catelyn
#> [15] Joffrey     --Sansa         Ned         --Pycelle
#> + ... omitted several edges
vcount(got)
#> [1] 127
ecount(got)
#> [1] 550
```

`UNW-` : non dirigé, et un nouvel attribut de nœud `sex (v/c)`.

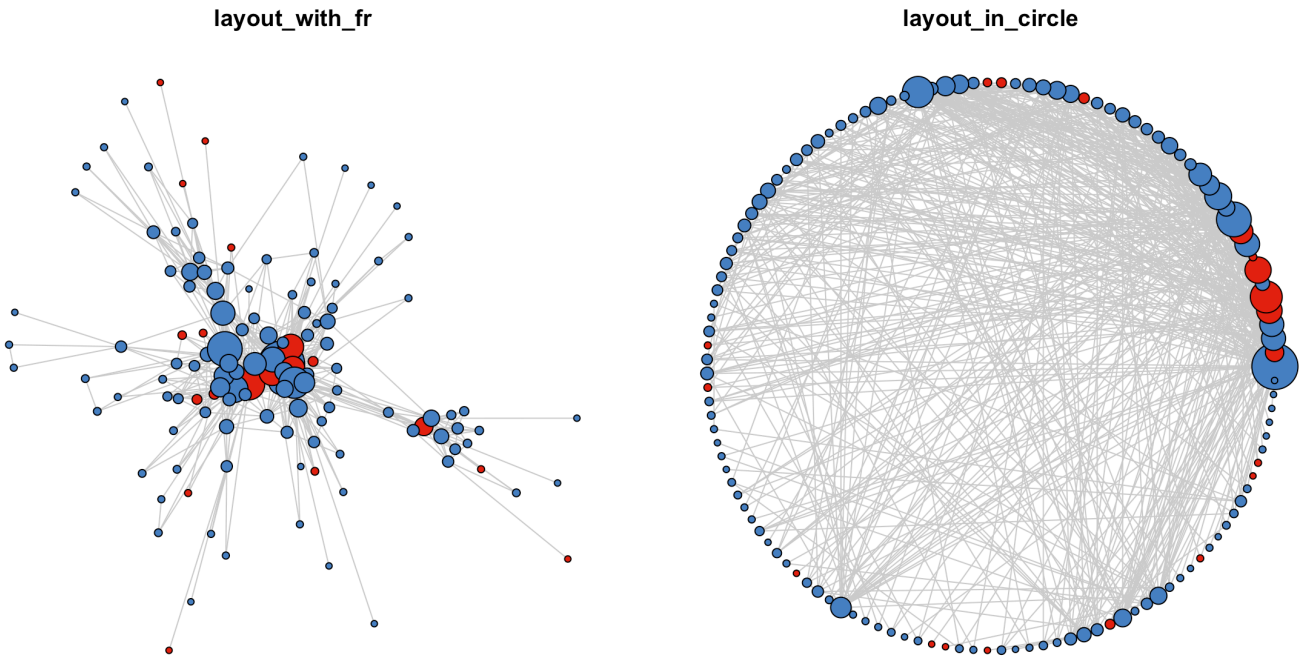
2. Les attributs

```
names(vertex_attr(got))
#> [1] "name" "sex"
names(edge_attr(got))
#> [1] "weight"
table(V(got)$sex)
#>
#>   F   M
#>  19 108
i <- which.max(E(got)$weight)
E(got)[i]
#> + 1/550 edge from 9f56d17 (vertex names):
#> [1] Ned--Robert
E(got)$weight[i]
#> [1] 192
```

`vertex_attr()` renvoie une **liste** (un élément par attribut, dans l'ordre de `V(got)`). 19 femmes pour 108 hommes. Le lien le plus lourd : Ned–Robert, 192 interactions.

3. Dessiner

```
couleurs <- if_else(V(got)$sex == "F", "#E30513", "#4A7FC1")
tailles <- 2 + degree(got) / 4
par(mfrow = c(1, 2), mar = c(0, 0, 1.5, 0))
set.seed(1)
plot(got, layout = layout_with_fr, vertex.color = couleurs, vertex.size = tailles,
     vertex.label = NA, edge.color = "grey80", main = "layout_with_fr")
plot(got, layout = layout_in_circle, vertex.color = couleurs, vertex.size = tailles,
     vertex.label = NA, edge.color = "grey80", main = "layout_in_circle")
```



Les **positions** changent ; le graphe (les nœuds, les liens, les degrés, les couleurs) est le même. Fruchterman-Reingold rapproche les nœuds liés : on devine des groupes (la cour et les Stark au centre, Jon et la Garde de nuit d'un côté, Daenerys et les Dothrakis de l'autre). Le cercle ne montre presque rien. `vertex.color` et `vertex.size` suivent l'ordre de `V(got)` : c'est pourquoi on les calcule à partir de `V(got)$sex` et `degree(got)`, pas à partir de `got_sexe`.

4. La matrice d'adjacence

```
A <- as_adjacency_matrix(got, sparse = FALSE)
dim(A)
#> [1] 127 127
A[1:4, 1:4]
#>             Ned Daenerys Jon Littlefinger
#> Ned           0        1   1            1
#> Daenerys      1        0   0            0
#> Jon           1        0   0            0
#> Littlefinger  1        0   0            0
isSymmetric(A)
```

```
#> [1] TRUE
all(rowSums(A) == degree(got))
#> [1] TRUE
sum(A)
#> [1] 1100
```

Une matrice 127 × 127, **symétrique** (graphe non dirigé), dont la somme des lignes est le degré. La somme totale, 1100, vaut deux fois le nombre de liens : chaque lien remplit deux cases.

```
W <- as_adjacency_matrix(got, attr = "weight", sparse = FALSE)
isSymmetric(W)
#> [1] TRUE
all(rowSums(W) == strength(got))
#> [1] TRUE
W["Ned", "Robert"]
#> [1] 192
mean(A == 0)
#> [1] 0.9317999
```

Avec les poids, la somme d'une ligne est la **force** (`strength()`), pas le degré. La matrice est vide à 93 % : c'est pour cela qu'igraph renvoie par défaut une matrice **creuse** (`sparse = TRUE`).

5. Le degré avec le tidyverse

```
deg_tidy <- got_aretes |>
  pivot_longer(c(from, to), values_to = "perso") |>
  dplyr::count(perso, name = "degre")
deg_tidy |> arrange(desc(degre)) |> head(5)
#> # A tibble: 5 × 2
#>   perso   degre
#>   <chr>   <int>
#> 1 Ned        57
#> 2 Tyrion     41
#> 3 Catelyn    37
#> 4 Robert     36
#> 5 Robb       30
all(deg_tidy$degre == degree(got)[deg_tidy$perso])
#> [1] TRUE
```

Chaque lien a **deux** extrémités : après `pivot_longer()` , chaque personnage apparaît une fois par lien. `count()` trie les noms par ordre alphabétique, `degree()` suit l'ordre de `V(got)` : on indexe donc par le **nom** (`degree(got)[deg_tidy$perso]`) avant de comparer. Comparer deux vecteurs dans des ordres différents donnerait un `FALSE` trompeur… ou pire, un `TRUE` par hasard.

Et si… ?

1. Dirigé par défaut

```
degree(g, "Ned", mode = "in")
#> Ned
#>   0
degree(g, "Ned", mode = "out")
#> Ned
#>  57
degree(g, "Ned")
#> Ned
#>  57
```

Ned n'a **aucun** lien entrant et 57 sortants : dans le fichier, il est toujours dans la colonne `from`. Cela reflète l'ordre dans lequel le fichier a été écrit, pas une propriété des personnages. Le degré total est juste, mais tout calcul qui suit les flèches (distances, centralités, communautés) serait faux, **sans message**.

2. Un nom mal écrit, un sexe mal placé

```
sexe_faute <- got_sexe |> mutate(name = if_else(name == "Ned", "Eddard", name))
graph_from_data_frame(got_aretes, directed = FALSE, vertices = sexe_faute)
#> Error in graph_from_data_frame(got_aretes, directed = FALSE, vertices = sexe_faute): S
```

Avec `vertices`, une **erreur** : c'est la bonne situation, on voit le problème.

```
g_sans <- graph_from_data_frame(got_aretes, directed = FALSE)
V(g_sans)$sex <- got_sexe$sex
head(V(g_sans)$name)
#> [1] "Ned"      "Daenerys" "Jon"      "Catelyn"  "Bronn"    "Cersei"
head(got_sexe$name)
#> [1] "Ned"          "Daenerys"     "Jon"          "Littlefinger" "Arya"
#> [6] "Catelyn"
sum(V(g_sans)$sex != V(got)$sex[match(V(g_sans)$name, V(got)$name)])
#> [1] 28
```

Sans `vertices`, les nœuds sont dans l'ordre d'**apparition** dans la liste d'arêtes, pas dans celui du fichier des sexes. L'affectation directe fonctionne, sans erreur ni message, et 28 personnages reçoivent le **mauvais** sexe. Si on tient à l'affecter après coup : `got_sexe$sex[match(V(g)$name, got_sexe$name)]`.

3. Un personnage sans lien

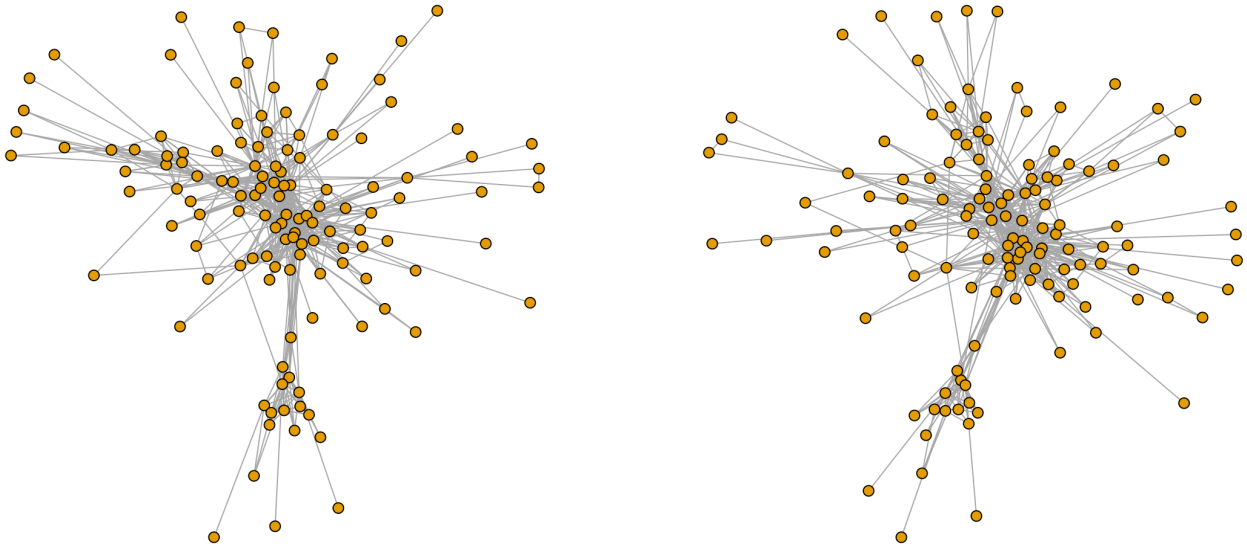
```
sexe_plus <- bind_rows(got_sexe, tibble(name = "Ghost", sex = "M"))
vcount(graph_from_data_frame(got_aretes, directed = FALSE, vertices = sexe_plus))
#> [1] 128
degree(graph_from_data_frame(got_aretes, directed = FALSE, vertices = sexe_plus), "Ghost"
#> Ghost
#>     0
```

```
vcount(graph_from_data_frame(got_aretes, directed = FALSE))
#> [1] 127
```

Avec `vertices`, Ghost est un nœud **isolé** (degré 0). Sans `vertices`, il n'existe pas : une liste d'arêtes ne connaît que les nœuds qui ont au moins un lien. Pour un réseau de gènes, ce sont les gènes sans aucune interaction qui disparaissent, et toute proportion calculée sur `vcount()` change.

4. Deux dessins sans graine

```
par(mfrow = c(1, 2), mar = c(0, 0, 0, 0))
plot(got, vertex.label = NA, vertex.size = 4)
plot(got, vertex.label = NA, vertex.size = 4)
```



Deux images **différentes** : la disposition par défaut part d'une position aléatoire. Aucune n'est plus « vraie » que l'autre ; la position n'est pas une donnée. Pour un rapport, `set.seed()` ou `l <- layout_with_fr(got)` calculé une fois et réutilisé.

5. Une matrice pondérée lue sans `weighted`

```
g_mat <- graph_from_adjacency_matrix(W, mode = "undirected")
g_mat
#> IGRAPH 23f64de UN-- 127 6525 --
#> + attr: name (v/c)
#> + edges from 23f64de (vertex names):
#>  [1] Ned--Daenerys     Ned--Daenerys     Ned--Daenerys     Ned--Jon
#>  [5] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#>  [9] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#> [13] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#> [17] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#> [21] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#> [25] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
```

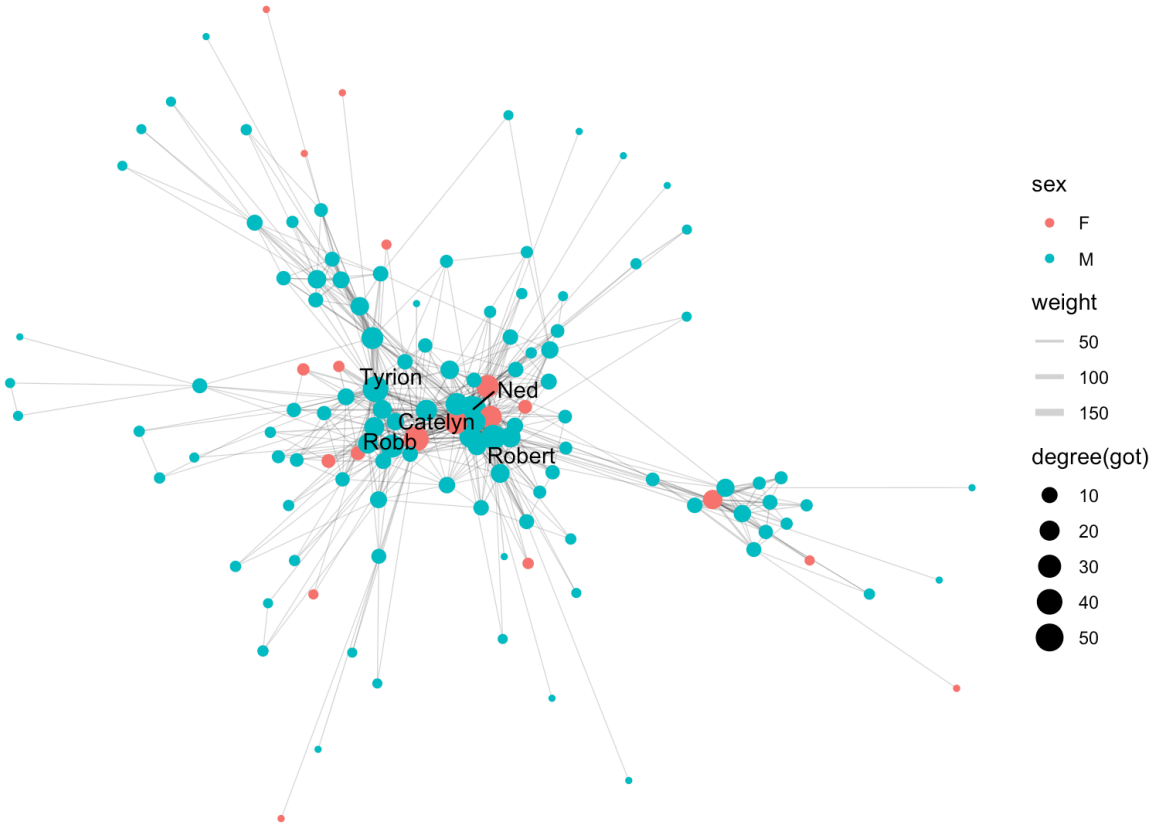
```
#> [29] Ned--Jon          Ned--Jon          Ned--Jon          Ned--Jon
#> + ... omitted several edges
ecount(g_mat)
#> [1] 6525
sum(E(got)$weight)
#> [1] 6525
count_multiple(g_mat, E(g_mat)[.from("Ned") & .to("Robert")])[1]
#> [1] 192
```

igraph 2.1.4 ne donne **ni erreur ni avertissement** : sans `weighted = TRUE`, chaque case est lue comme un **nombre de liens**. On obtient un multigraphe de 6525 liens (la somme des poids), sans attribut `weight` (pas de `W` dans l'en-tête), avec 192 liens parallèles entre Ned et Robert. Le degré de Ned devient 1290 au lieu de 57. Avec `weighted = TRUE` :

```
graph_from_adjacency_matrix(W, mode = "undirected", weighted = TRUE)
#> IGRAPH 5521a16 UNW- 127 550 --
#> + attr: name (v/c), weight (e/n)
#> + edges from 5521a16 (vertex names):
#>  [1] Ned--Daenerys       Ned--Jon            Ned--Littlefinger
#>  [4] Ned--Arya           Ned--Catelyn        Ned--Cersei
#>  [7] Ned--Joffrey        Ned--Sansa          Ned--Tyrion
#> [10] Ned--Jeor           Ned--Robb           Ned--Bran
#> [13] Ned--Jaime          Ned--Loras          Ned--Jory Cassel
#> [16] Ned--Jorah          Ned--Ros            Ned--Benjen
#> [19] Ned--Greatjon Umber Ned--Pyp            Ned--Renly
#> [22] Ned--Barristan      Ned--Hound          Ned--Theon
#> + ... omitted several edges
```

Pour aller plus loin : `ggraph`

```
library(ggraph)
set.seed(1)
ggraph(got, layout = "fr") +
  geom_edge_link(aes(edge_width = weight), alpha = 0.2) +
  geom_node_point(aes(colour = sex, size = degree(got))) +
  geom_node_text(aes(label = if_else(degree(got) >= 30, name, "")), repel = TRUE) +
  scale_edge_width(range = c(0.2, 2)) +
  theme_void()
```



La largeur des liens montre les relations fortes (Ned–Robert, Daenerys–Jorah) que le dessin non pondéré traite comme les autres. `facet_nodes(~ sex)` dessine un panneau par sexe, en gardant les mêmes positions.